Ledbetter Data Pump (LDP)

Architecture White Paper

A High-Performance, YAML-Driven Data Movement Engine

Version 1.0 • November 2026 • Status: Ready for Implementation

Ledbetter Data Pump (LDP)

Architecture White Paper

A High-Performance, YAML-Driven Data Movement Engine

Version 1.0

Date: November 2026

Status: Ready for Implementation

1. Executive Summary

Ledbetter Data Pump (LDP) is a modern, high-throughput data movement engine designed as a clean replacement for the SSIS data flow (the “data pump”). It moves and lightly transforms data between relational databases and files at high concurrency while remaining simple to configure, version-control, and operate.

Core principles:

• Rust execution engine for performance, safety, and fearless concurrency

• Declarative YAML job files (no GUI layout metadata, no XML bloat)

• Partitioned, shared-nothing execution with Arrow columnar buffers

• Production-grade idempotency, checkpointing, and error handling

• Thin optional Python control plane / bindings

Primary targets: SQL Server, Oracle, PostgreSQL, Snowflake, CSV/TSV, Parquet.

Performance goal: 10–20+ concurrent pipelines, hundreds of thousands to >1M rows/sec on modest hardware for bulk loads.

2. Goals and Non-Goals

Goals

• High-throughput bulk and incremental data movement

• Safe restarts (idempotent) after mid-job failure

• Clean, human-readable, Git-friendly job definitions

• Cross-platform single binary (Linux, macOS, Windows)

• Extensible connector model

• Observability (structured logs, metrics, audit trail)

Non-Goals (v1)

• Full SQL query engine or complex multi-table joins

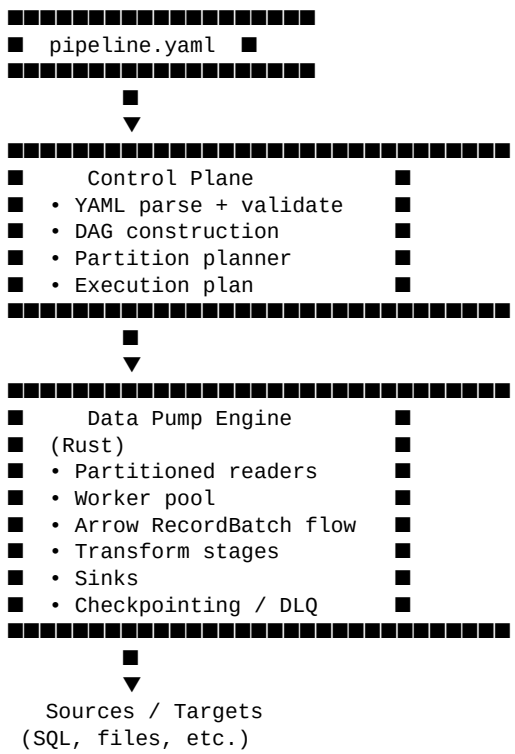
• Real-time streaming / CDC (batch and incremental batch only)

• Graphical designer

• Distributed multi-node execution (single-process multi-threaded is sufficient)

• Arbitrary user-defined functions in Python/Rust at runtime (later)

3. High-Level Architecture



Two clean layers:

1. **Control Plane** — configuration, validation, planning

2. **Data Pump Engine** — pure execution

4. Control Plane

Responsibilities:

• Parse and validate `pipeline.yaml` against a strict schema

• Build a directed acyclic graph (DAG) of stages

• Plan partitions (range, hash, file offsets, etc.)

• Inject watermarks for incremental loads

• Produce an immutable **Execution Plan** handed to the engine

• Manage run state and final reporting

Can be implemented entirely in Rust (preferred for a single binary) or with a thin Python wrapper for orchestration.

5. Data Pump Engine

5.1 In-Memory Data Model

All data flows as **Apache Arrow `RecordBatch`**.

Benefits:

• Columnar layout → excellent CPU cache behavior

• Strong typing and schema evolution control

• Zero-copy potential between stages and to Parquet

• Mature ecosystem (`arrow-rs`)

A `DataChunk` is simply:

```
pub struct DataChunk {
    pub chunk_id: u64,
    pub batch: RecordBatch,
}
```

5.2 Execution Model

• **Partitioned reading**: source is split into independent ranges/offsets so multiple threads read concurrently without coordination.

• **Shared-nothing workers**: each worker owns its chunk from read → transform → buffer → sink.

• **Bounded channels** (`crossbeam-channel`) provide backpressure.

• **Rayon** thread pool (or hybrid with Tokio for I/O) for scheduling.

Typical pipeline stages per worker:

1. Read partition → `RecordBatch`

2. Apply filter / derive / rename (vectorized where possible)

3. Conditional split (if configured)

4. Write to sink (batched)

5.3 Concurrency Recommendation

• Use **async I/O** (`tokio` + `sqlx` or ADBC) for database readers/writers.

• Use **rayon** for CPU-bound transform stages.

• Bounded channels between stages to control memory.

6. YAML Job Specification

Full declarative schema. Example (abridged):

```
version: "1.0"
pipeline:
  name: "order_sync"
  description: "Postgres → Snowflake + Parquet"

  settings:
    max_threads: 16
    buffer_size_mb: 128
    error_handling:
      strategy: "dlq_and_continue"   # fail_fast | dlq_and_continue | drop_row
      dlq_path: ".ldp/dlq_orders.jsonl"
      max_bad_rows: 100
      max_error_percentage: 0.5
    failure_policy:
      mode: "resume_from_checkpoint" # fail_fast | resume_from_checkpoint
      state_file: ".ldp/state_orders.json"

  resources:
    connections:
      - name: "src_pg"
        type: "postgres"
        url: "env://PROD_PG_URL"
        pool_size: 8
      - name: "tgt_sf"
        type: "snowflake"
        url: "env://SNOWFLAKE_URL"

  dataflow:
    sources:
      - id: "orders"
        connection: "src_pg"
        query: |
          SELECT order_id, customer_id, region, amount, status, updated_at
          FROM raw_orders
          WHERE order_date >= '2026-01-01'
        partitioning:
          type: "range"          # range | hash | none
          column: "order_id"
          chunks: 16
        incremental:
          mode: "watermark"
          column: "updated_at"
          initial_value: "2026-01-01T00:00:00Z"
```

```
transformations:
  - id: "filter_valid"
    input: "orders"
    type: "filter"
    condition: "status != 'CANCELLED' AND amount > 0"

  - id: "enrich"
    input: "filter_valid"
    type: "derive"
    fields:
      - name: "taxed_amount"
        expression: "amount * 1.0825"
      - name: "processed_at"
        expression: "now()"

  - id: "split_region"
    input: "enrich"
    type: "conditional_split"
    branches:
      - name: "na"
        condition: "region IN ('US','CA','MX')"
      - name: "intl"
        condition: "region NOT IN ('US','CA','MX')"

targets:
  - id: "sf_na"
    input: "split_region.na"
    connection: "tgt_sf"
    table: "stg_orders_na"
    mode: "atomic_swap"       # append | truncate | merge | atomic_swap
    primary_key: ["order_id"]
    batch_size: 10000

  - id: "parquet_intl"
    input: "split_region.intl"
    type: "parquet"
    path: "/data/lake/intl_orders/"
    compression: "zstd"
    partition_by: ["region"]
```

Supported Stage Types (v1)

• **Sources**: SQL (with partitioning + optional watermark), CSV/TSV, Parquet

• **Transformations**: filter, derive (simple expressions), rename, drop, conditional_split

• **Targets**: SQL (append, truncate, merge, atomic_swap), CSV, Parquet

7. Idempotency & Recovery

Three complementary mechanisms:

1. **Chunk Checkpointing**

After a partition is successfully written, its `chunk_id` is recorded in the state file. On restart, completed chunks are skipped.

2. **Atomic Staging & Swap** (database targets)

- Write all data into a staging table

- On full success: atomic rename / swap / merge into production

- On failure: staging is discarded; production is untouched

3. **Deterministic Upsert / Merge**

For incremental or large tables where full swap is impractical. Requires a declared primary key.

State file format (JSON or SQLite):

```
{
  "pipeline": "order_sync",
  "run_id": "2026-11-10T17:55:00Z",
  "completed_chunks": [0, 1, 3, 4],
  "watermark": "2026-11-10T12:34:56Z"
}
```

8. Error Handling

• Row-level isolation: bad rows do not crash the worker.
• Configurable strategy: `fail_fast`, `dlq_and_continue`, `drop_row`.
• Error budget (`max_bad_rows` or percentage). Exceeding the budget aborts the job cleanly.
• Dead Letter Queue written as JSONL (or Parquet) with original row + error reason.

9. Connectors

Interface (trait):

```
trait Source {
    async fn read_partition(&self, partition: Partition) -> Result<RecordBatch>;
}

trait Sink {
    async fn write_batch(&self, batch: RecordBatch) -> Result<()>;
    async fn finalize(&self) -> Result<()>;  // for atomic swap, close files, etc.
}
```

Initial connectors to implement:

1. PostgreSQL (range partitions via `sqlx`)

2. Parquet (arrow + parquet crate)

3. CSV/TSV (memory-mapped or streaming)

4. Snowflake (ADBC or official connector)

5. SQL Server

Type mapping is centralized in a small type system that maps Arrow types ↔ database types.

10. Observability

Structured JSON logs to stdout (easy to ship to Loki, CloudWatch, etc.):

```
{
  "timestamp": "2026-11-10T17:55:32Z",
  "level": "INFO",
  "pipeline": "order_sync",
  "event": "chunk_completed",
  "chunk_id": 7,
  "rows": 25000,
  "duration_ms": 142,
  "rows_per_sec": 176056
}
```

Additional metrics: rows read/written, errors, active threads, memory, final run summary.

11. Optional Python Integration

• **PyO3** bindings for the engine so Python can:

- Submit jobs

- Receive Arrow batches (via `pyo3-arrow`)

- Orchestrate multiple pipelines

• Control plane can also be written in Python if desired; the engine remains Rust.

• GIL is released during all heavy work.

12. Crate / Module Structure

```
ldp/
■■■ ldp-config/          # YAML schema, validation, DAG, partition planner
■■■ ldp-core/            # DataChunk, worker pool, channels, checkpointing, DLQ
■■■ ldp-arrow/           # Arrow helpers, type mapping
■■■ ldp-connectors/
■   ■■■ postgres/
■   ■■■ snowflake/
■   ■■■ sqlserver/
■   ■■■ parquet/
```

```
■   ■■■ csv/
■■■ ldp-cli/            # Binary entry point
■■■ ldp-python/         # Optional PyO3 crate
```

13. Key Design Decisions

14. Implementation Roadmap

Phase 1 – Core Engine (2–3 weeks)

• YAML schema + validation

• Arrow-based DataChunk

• Worker pool + channels

• Simple filter + derive

• Checkpointing

• Postgres source + Parquet sink

• CLI

Phase 2 – Production Hardening

• Atomic staging/swap

• DLQ + error budgets

• Watermark incremental loads

• Structured logging + run summary

• CSV connector

Phase 3 – Broader Connectors & Polish

• Snowflake, SQL Server

• Conditional split

• Python bindings

• Metrics endpoint (Prometheus optional)

Phase 4 – Advanced (optional)

• DataFusion integration for richer expressions/joins

• More sophisticated partitioning strategies

15. Example Minimal Job

```yaml
version: "1.0"
pipeline:
  name: "simple_copy"
  settings:
    max_threads: 8
  resources:
    connections:
      - name: "pg"
        type: "postgres"
        url: "env://PG_URL"
  dataflow:
    sources:
      - id: "src"
        connection: "pg"
        query: "SELECT * FROM customers"
        partitioning:
          type: "range"
          column: "id"
          chunks: 8
    targets:
      - id: "tgt"
        input: "src"
        type: "parquet"
        path: "./output/customers/"
```

16. Conclusion

This architecture delivers a modern, high-performance data pump that is:

• Dramatically simpler to author and maintain than SSIS

• Significantly faster and more resource-efficient

• Safe to restart

• Ready to implement immediately with well-known Rust crates

The design deliberately stays focused. Build the core (Arrow + partitioned workers + YAML + checkpointing) first. Expand connectors and transform power only after the engine is proven.

This document is the implementation contract.

End of White Paper